PART 4 – INTRODUCTION TO PROCESS FLOW

This fourth part of the Primer focuses on developing complex logic using *FlexSim*'s logic builder called Process Flow. Process Flow enables users to define and specify logic via a flowchart-like, drag-and-drop interface and link the logic to 3D objects.

After introducing the basic concepts of Process Flow, and defining its modeling environment, this section describes modeling in Process Flow via two examples, both building upon the basic Primer model (finishing and packing containers). The two examples:

1. Implement a reorder-point inventory system for the components that are packed into containers.
2. Create a custom task sequence for the finish operator.

Both examples are not that complex, but they are complex enough that they cannot be implemented via the drop-down menu options on the 3D objects. In the past, and with many other simulation software, the only way to model these system behaviors is via custom coding. In the case of *FlexSim*, this would be via FlexScript. While the logic can still be implemented via FlexScript, Process Flow enables the modeling without coding and makes the logic more transparent. Transparency is enhanced through Process Flow since all of the logic is contained in one place, on the Process Flow interface, and not dispersed in 3D objects. In addition, the logic is more transparent because the logic is represented in an easy to understand flowchart-like format.

Process Flow, like the 3D aspects of *FlexSim*, is a very robust and powerful modeling environment. Therefore, since this primer is intended only to provide a basic introduction to *FlexSim*, only a few features and capabilities are described. However, this should provide a good starting point and a solid foundation for modeling with Process Flow.

While the focus is on Process Flow, in both examples various aspects of the 3D model are revisited and additional 3D features are introduced.

1 BASIC CONCEPTS

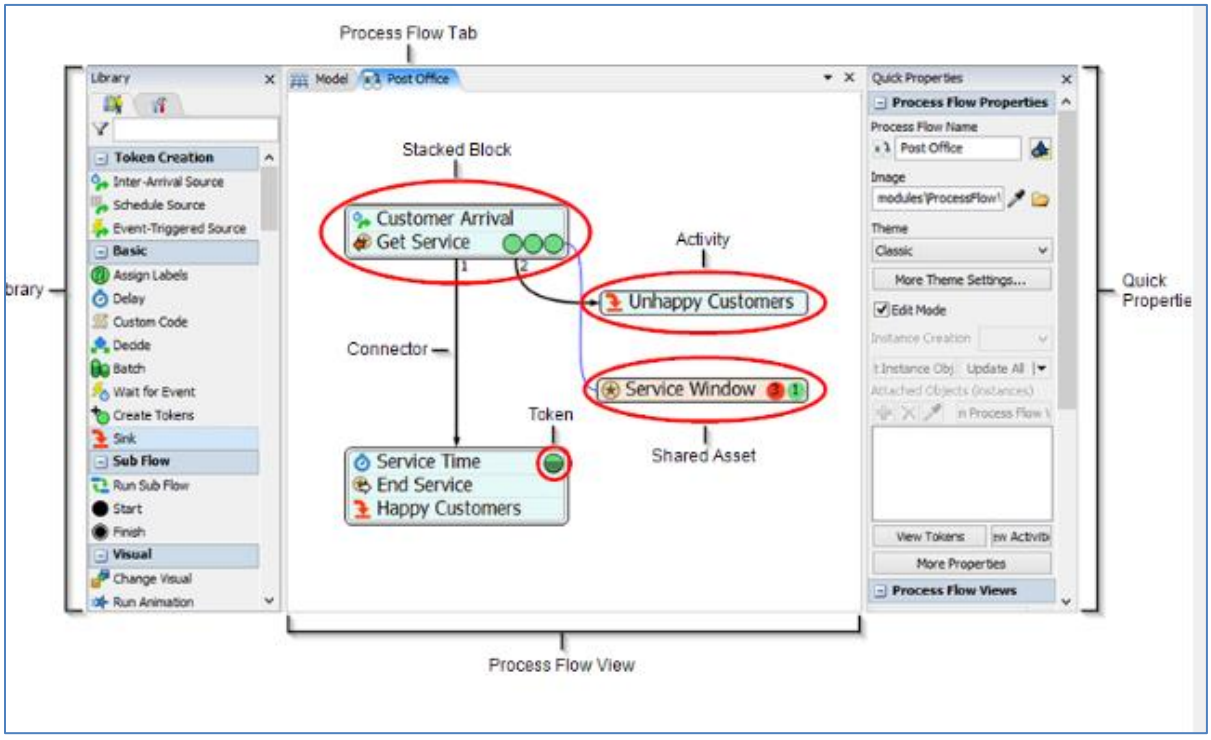
This first section introduces the basic elements of Process Flow and its modeling environment. While the interface and library of modeling tools are different than in 3D, it operates in a similar manner. As with 3D objects, Process Flow activities are dragged onto a modeling surface and are connected to represent the way a system behaves. This section also describes the basic operation of a reorder-point inventory system.

1.1 Basic Process Flow concepts and modeling environment

Process Flow is a part of *FlexSim* that is used to add complex processing logic into a simulation model. The logic is used to define interactions among *FlexSim* objects and other elements as a simulation runs. While *FlexSim* provides a lot of logic-modeling capability through its drop-down menu options on 3D objects and tools, there is a limit as to how much can be pre-specified. Typically, custom logic requires scripting or writing computer code to implement the more complex logic. This can be done through *FlexSim*'s programming language, called FlexScript, a subset of C++. However, Process Flow, reduces the need for programming by using a set of logic-building activities that can be defined and combined via a flowcharting type of paradigm.

As with any modeling endeavor, it is always best to build the logic in steps.

Process Flow has its own logic-building interface, analogous to the 3D modeling surface, and its own set of modeling objects, referred to as *activities*. The basic components of the Process Flow modeling environment and their terminology are shown in the figure below. The Process Flow View is accessed via its icon in the Main Menu or via the Toolbox. The logic is built by selecting and combining activities.



Activities are the building blocks of Process Flow - they are logical operations or steps in a logical process. Activities are analogous to objects in 3D since they are dragged from their library onto a Process Flow modeling

surface or workspace. Also like 3D objects, activities have properties that define their behavior. Activities are grouped into the following categories. This primer only addresses a subset of the activities, but the full list is provided so that the reader is aware of the capabilities of Process Flow.

- *Token Creation* activities are analogous to a Source in 3D. Tokens can be created by inter-arrival times, schedule, and by "listening" for specific events to occur in a 3D model.
- *Basic* activities include assigning labels, implementing a delay in a flow, waiting for an event to occur in a 3D model, making decisions, destroying tokens, etc.
- *Sub Flow* activities are used to create a separate process flow that contains logic that may be used by multiple objects or activities; it is analogous to a function or subroutine in computer programming.
- *Visual* activities change the appearance of an object in the 3D model or triggers an animation on a 3D object.
- *Object* activities are used to create, move, or destroy objects such as flowitems, fixed resources, task executers, etc. in a 3D model.
- *Task Sequences* activities are used to build custom task sequences that are assigned to task executers in a 3D model, including travel, load, unload, delay, etc.
- *Shared Assets* activities are used to define, access, and manage Lists, Resources, Variables, and Zones (keep statistics for a group of activities and restrict access to activities based on token properties).
- *Coordination* activities manage relationships between multiple tokens in a process flow, including splitting, joining, and synchronizing tokens.
- *Preemption* activities are used manage interruptions in a token's flow.
- *Display* activities are used to annotate a process flow with text, arrows, and images; they do not affect the logic.
- *Flowchart* activities are shapes that can help organize and visualize a process flow; like Displays, they do not affect the logic.
- *People Activity Sets* are preconfigured activities that are bundled together that model a basic People Module task, such as Walk then Process, Escort then Process, Wait then Process, etc.
- *People Basic* are activities that create or delete a Person flowitem or process a token.
- *People Resources* are activities that facilitate acquiring and releasing resources (Locations, Staff, Transports, and Equipment) in a People-based model.
- *People Sub Flows* are activities that are tied to pre-built Sub-Flow objects.

A **token** is the basic component of Process Flow logic. Analogous to flowitems in 3D, tokens flow through activities as a simulation runs. *Tokens are typically more abstract than flowitems since they usually represent logic flow, rather than physical flow.* Each token has a unique ID and a set of labels or characteristics. Tokens are depicted as green circles in Process Flow as a simulation runs; they may change color depending on their use and state.

Tokens move from activity to activity either by **connectors** or through **blocks**. A block is a set of activities that have been "snapped" together; i.e., activities that form a single sequence of process flow steps.

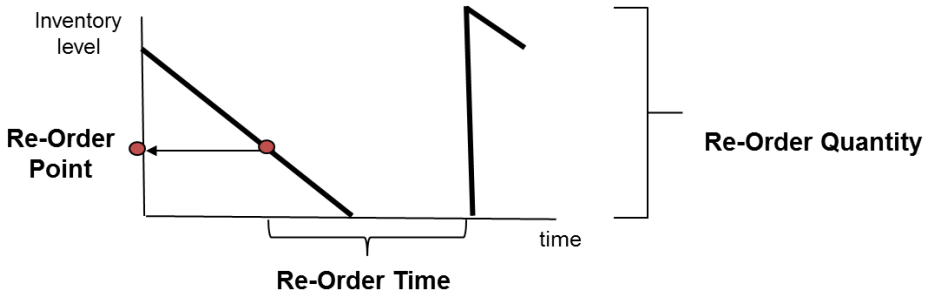
Tokens have **Labels**, which are an important part of modeling with Process Flow. Labels store information that is used in the logic. As in the 3D objects, each label has a name and value; the value can be of any data

type, e.g., numbers, text, arrays, object references, etc. Token labels are specified in the dot-notation format token.LabelName, e.g. token.Type.

Begin with the basic model from the previous section, named Primer_3-6b, and **Save As** Primer_4-1. Basically make a copy of the model so that it can be customized and the original model is retained.

1.2 Using Process Flow to model inventory policy

Continuing the primer example from the previous section, Process Flow is used to model a reordering system for supplying components to the packing area. Rather than having components delivered in batches on a schedule, as in the current model, components are ordered when their inventory level falls below a specified reorder point. As shown in the figure below, when the inventory level falls below the specified reorder point (ROP), an order is triggered for a specified quantity (ROQ) of the component and that quantity is delivered as a batch after a specified time (ROTime).



Therefore, three new parameters must be specified for each component: ROP, ROQ, and ROTime.

- ROQ is analogous to the batch size.
- ROTime needs to include ordering, fulfilling, and delivery times.
- ROP is specified as a quantity, level of inventory, and not time.

Another parameter that is considered in the model is the initial inventory (IInv), the number of components on hand when a simulation starts. It can be any value above zero. However, if the initial value is below the reorder point, the logic would need to ensure an order is placed when the simulation starts. For simplicity, *assume the initial inventory value is above the reorder point when a simulation starts.*

These additional parameters need to be added to the model.

➢ Update the Global Table named Components as shown in the figure to the right.
  - The time between batches will no longer be used since it will be replaced by the reorder point system. However, leave the entry in the table in case there is a need to return to the scheduled-order system. In fact, some components might use the scheduled-order system and some might use the reorder-point system.

Model | ProcessTimes | ProductMix | Packing | Components

	CompA	CompB
Time between batches	60	30
Batch size (Order Quantity)	24	60
Time to unpack a batch - fixed (minutes)	1	1
Time to unpack a batch - per item (minutes)	0.05	0.05
Initial Inventory	12	30
Reorder Point	6	20
Time to receive an order	120	120
Flowitem ID	10	11

- The former batch size (row 2 in the table) will now be the component's order quantity. Change the values of the Batch size from their previous values of 3 and 5 to **24** and **60**, respectively. Update the text in the row header to indicate the values are batch sizes.
- Four new rows are added to the table for: initial inventory, reorder point, time to receive an order and flowitem ID. Each of these are explained when used in the sections below; therefore, just enter them in the table for now, as shown above.

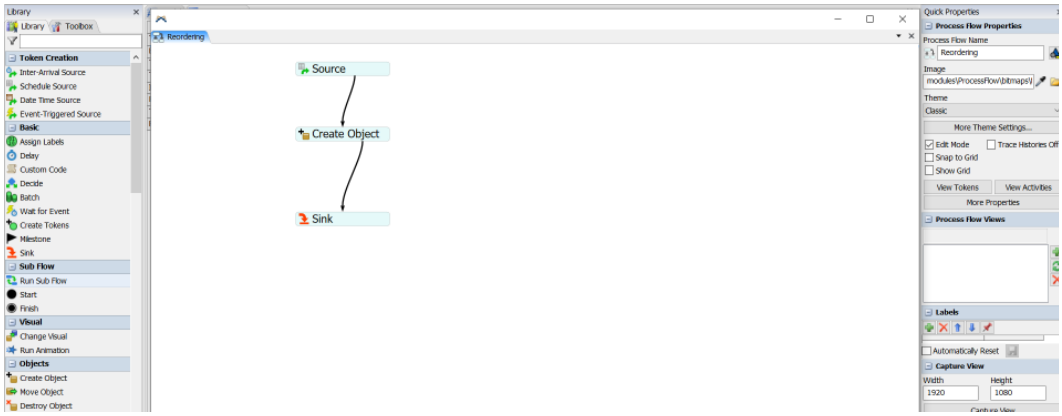
2 MODELING INITIAL INVENTORY

A very simple extension of the model is used to introduce modeling with Process Flow.

➢ Through the Main Menu icon, or via the Toolbox, select **Add a General Process Flow**, and change its name from the default ProcessFlow to **Reordering**.

➢ As shown in the figure below, drag out three activities onto the Process Flow modeling surface:
- Schedule Source (from the Token Creation section of the Activity Library)
- Create Object (from the Objects category)
- Sink (from the Basic category)

➢ Link the activities using the Connector – move the mouse over an activity until the mouse cursor changes to a chain, then holding down the left mouse button, drag the arrowed-line connector to the other activity. The activities may be positioned by selecting them with the left mouse button and dragging them to the desired position.

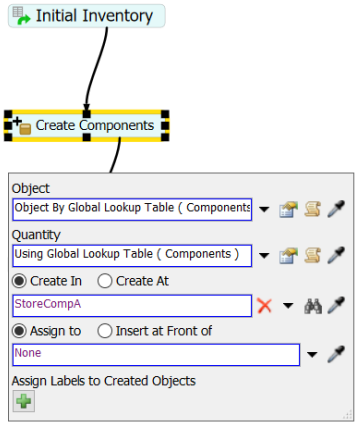


For now, only changes are made to the properties of the Create Object activity.

➢ However, change the name of the Schedule Source activity from Source to **Initial Inventory** and change the name of the Create Object activity from Create Object to **Create Components**. To do this select the activity and change its property values in the **Activity Properties** section of the Quick Properties window.

Activity properties are changed either by: (1) selecting the activity and editing in the Quick Properties window or (2) clicking on the icon for the activity located in the left-hand portion of the activity's flowchart shape.

The Create Object activity creates flowitems that represent the components and puts them into their Queue in the packing area. *Note that the flowitems are created directly in the Queue and do not come from the Source.* As a result, these flowitems do not pass through the Queue's OnEntry trigger. Therefore, any logic on the OnEntry trigger would not be invoked for these flowitems.



- Change the Create Object activity as described below and as shown in the figure to the right.
  - The value of the **Object** property is obtained by selecting the drop-down menu option **Flowitem By Global Table Lookup**.
  - **Table Name**: **Components** is selected from the drop-down menu.
  - **Row**: **8**, based on the revised Global Table above.
  - **Column**: **1**; this is only for the first component - it will be changed later to a more general setting.

In this case, Row 8, Column 1 of the Global Table contains the value 10; this references the 10th entry in the Flowitem Bin, which is the flowitem created earlier named CompA.

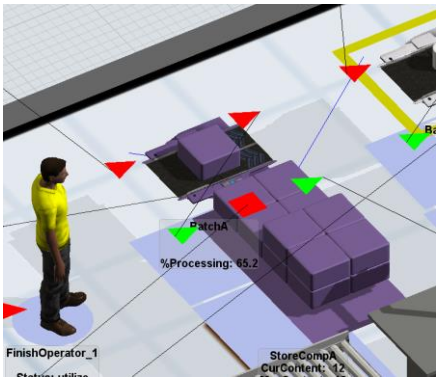
- The **Quantity** property in the Create Object activity is obtained using the drop-down menu option **By Global Table Lookup**. Values for the properties are:
  - **Table: Components** from the drop-down menu
  - **Row: 5**, since the initial inventory values are stored in this row in the table
  - **Column: 1**, for the first component; as indicated earlier, this will be changed later to a more general setting.

The **Creates In** property in the Create Object activity needs to refer to the component's Queue. The default model() is changed to the Queue by using the Sampler tool (pipette or eyedropper icon).

- Select the Sampler and use it to select the **StoreCompA** object in the 3D view. A more general means of setting this will be described later.

The **Assign to** property can be left as is or its value deleted, resulting in the value **None**.

- **Reset** and **Run** the model. At time 0 the inventory for CompA should appear as in the figure to the right – there are 12 purple boxes in the StoreCompA object and one batch of components at the Separator for the Operator to unpack. To obtain this view, you must be very quick on clicking between Run and Stop on the Execution toolbar. Otherwise, the Operator will have unpacked the batch and the Queue will have an additional 24 items.



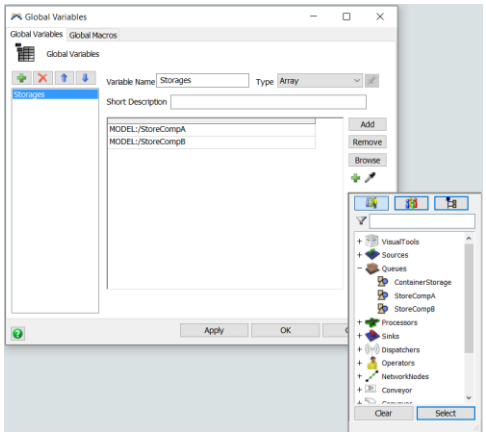
The **Step** control, next to **Stop** on the Execution toolbar, can be used to move through a model's execution one event at a time. This is better than trying to start and stop a model quickly. It will likely take several clicks of **Step** for the items to appear since *FlexSim* executes multiple events at simulation time zero.

If you haven't already done so, save the model. Recall, it is good practice to save often.

The modeling steps defined above need to be repeated for CompB. However, initializing the inventory can be generalized so that it occurs for any number components that are included in a model. To do so, the *FlexSim* tool Global Variables is introduced.

Global Variables are user-defined information that can be accessed from anywhere in a *FlexSim* model. The variables can be of various types such as Real (e.g., 123.45), Integer (e.g., 12), String (e.g., ABcdE), etc. In this case, the Global Variable is of the type Array, which means the variable contains an indexed ordered set of values. For example, for the arrayed variable myVAR = (12, 25, 17), myVAR(1) = 12, myVAR(2) = 25, etc.

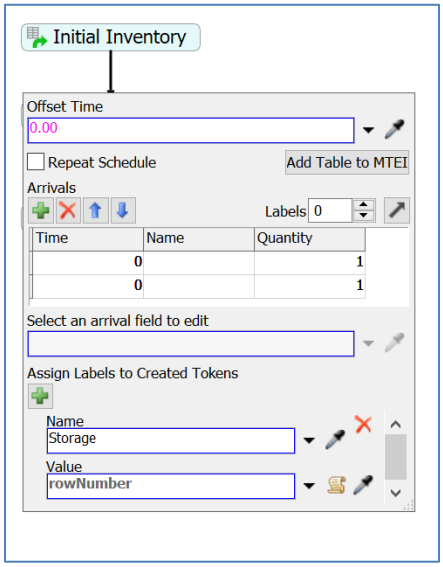
- Create an arrayed Global Variable named Storages via the Toolbox – first select **Modeling Logic**, section of the drop-down menu, then select **Global Variable**. As shown in the figure to the right:
  - Change the **Variable Name** from variable1 to **Storages**.
  - Using the drop-down menu, change the **Type** property from the default Integer to **Array.**
  - The elements of the array can be added by using the + drop-down menu or the Sampler, both tools are located below the Browse button on the interface. The figure to the right shows the object selection menu. By either method, add the two components' storage objects, **StorageCompA** and **StorageCompB**.



The data have been structured so that the i^{th} component in the packing area is represented as both the i^{th} column in the Global Table Components and the i^{th} element in the Global Variable Storages. Therefore, initial inventory

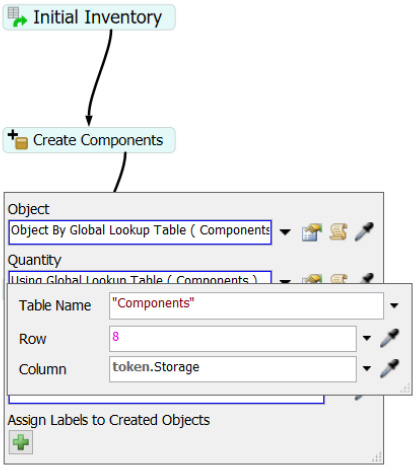
can be created for each component in a general manner by cycling through all of the components. This concept is now implemented in the Initial-Inventory Process Flow activity.

The Schedule Source activity is modified as follows and as shown in the figure to the right.



- The **Offset Time** remains at the default value **0.0** since the initial inventory is added at the start of the simulation.
- One arrival to the process flow is needed for <u>each</u> component; i.e., a token is created for each component that is used in the packing area. Therefore, a row is added to the **Arrivals** table in the Initial Inventory activity. This is done with the + button. Both arrivals are the same; they occur at **Time** value **0.0** and the **Quantity** is **1**, one token per component.
- A label is added to each token by clicking the + button at the bottom of the interface.
  - Change the **Name** property from the default labelName to **Storage**.
  - Change the **Value** property from the default 0.0 to **Arrival Row Number**, an option selected from the drop-down menu. When selected, *FlexSim* enters the command **rowNumber** in the **Value** cell. Therefore, each token will have a label that corresponds to the row number in the arrival table above. This will be used as an index for both the Global Table and the Global Variable. In this case, the indexes are 1 and 2 for CompA and CompB, respectively.

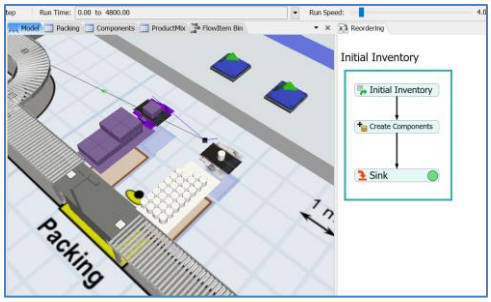
The index is used in the Create Components activity for each component. The activity is modified as follow:



- For the property **Object**, its **Column** value is changed to **token.Storage**; i.e. the column in the Global Table is the index or the number of the component. As shown in the figure to the right, this is accomplished through drop-down menu options in the property **Columns** by first selecting **Labels**, then **Storage**.
- Similarly, for the property **Quantity**, its **Column** value is also changed to **token.Storage** in the same manner as above. The column in the Global Table is the index of the number of the component.
- For the property **Create In**, use the Global Variable Storage and replace the single specific object reference with **Storages[token.Storage]**. There is no drop-down menu option for Global Variables so this needs to be typed in, but *FlexSim*'s context-sensitive help facilitates the entry.
- Create a label on the component flowitem by using the + button in the **Assign Labels to Created Objects** section of the interface. The property values are:
  - Change the **Name** from the default labelName to **Type**.
  - Change the **Value** from the default 0.0 to **token.Storage**, obtained from the drop-down menu **Token Label**.

Tidy up the Process Flow logic. Select the **Process** shape from the Flowchart category on the Activity menu and click near the three activities. The shape's box will surround the activities. As shown in the figure below, reshape the box border and name the shape Initial Inventory. This section of the Process Flow logic can be easily moved as a group.

- **Reset** and **Run** the model. At time 0 the inventories for CompA and CompB should appear as in the figure to the right – there are 12 purple boxes in the StoreCompA object and 30 white cylinders in the StoreCompB object, both of which are from the initial inventory logic just implemented in Process Flow. Note in the Process Flow logic in the right panel, the second token is about to exit.



If you haven't already done so, save the model. Recall, it is good practice to save often.

Now, for each component that might be added in the future, all that needs to be changed are to add a(n):

- column in the Global Table Components with the property values specified,
- row in the Arrivals table in the Initial Inventory Process Flow activity, and
- element in the arrayed Global Variable Storages referencing its Queue.

The above logic only executes once, at the beginning of a simulation whenever a model is Reset. However, it provides the basis for the reorder-point inventory system that is modeled in the next section.

3 MODELING INVENTORY REORDER POLICY

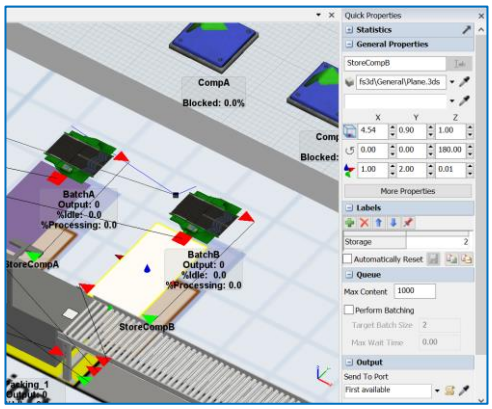
This section defines the reorder-point inventory system logic and describes in detail how to implement it using Process Flow. However, before doing so, a few preparatory changes are made in the 3D model. Why these changes are being made is explained as they are used in the next section.

Begin with the basic model from the previous section, named Primer_4-1, and **Save As** Primer_4-2. Basically make a copy of the model so that it can be customized and the original model is retained.

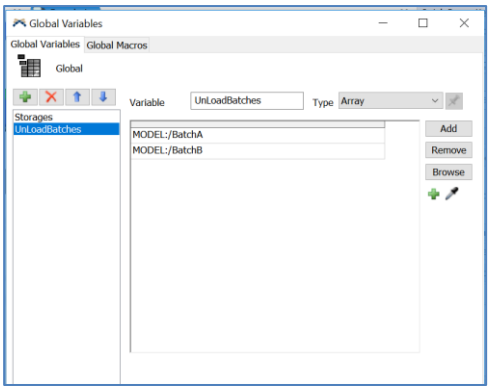
3.1 Changes in 3D objects for use in Process Flow

Before extending the Process Flow logic, the following preparatory changes are made to the model.

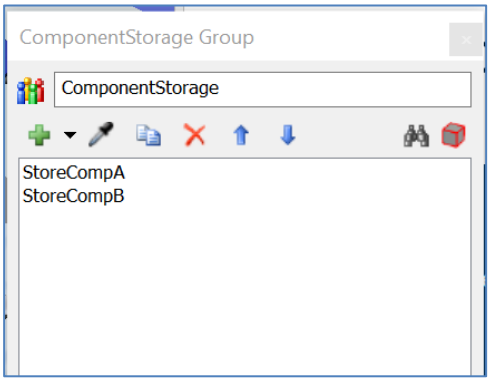
- Create and set a number label for each component's Queue that indicates what component it stores. Name the label **Storage** and set its value to **1** for StoreCompA and **2** for StoreCompB. Therefore, the item (component) with the label Type=1 is stored in the Queue with the label Storage=1, the component with the label Type=2 is stored in the Queue with the label Storage=2, etc. The label is shown in the Quick Properties view for StoreCompB in the figure to the right.



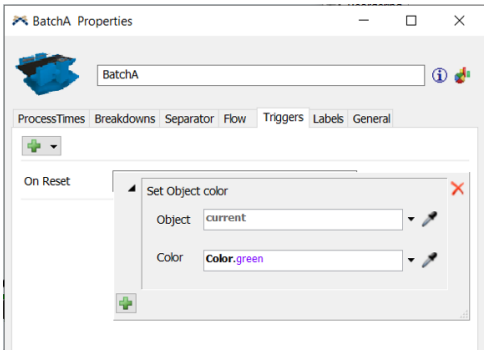
- Create another arrayed Global Variable, named **UnLoadBatches**. As shown in the figure to the right, it contains pointers to the Separator objects, BatchA and BatchB. It is created in the same way as the Global Variable Storages.



- Create a Group, as shown in the figure to the right. As the name indicates, the Group tool is a way to process similar items in a like manner.
  - Create the Group either via the Toolbox or by right-clicking on an object and add it to a current group or add it to a new group. In this case, use the Toolbox to create a Group.
  - Change its name from Group1 to **ComponentStorage**.
  - Add the two component storage Queues, StoreCompA and StoreCompB by using either the Sampler or the + button and then Select Objects menu.



- Set the color of the Separators, where the batches are unloaded to green. Later they will be changed to red when they are in they are awaiting an order. As shown in the figure to the right:
  - On the Triggers tab, add an **On Reset** trigger, on both objects BatchA and BatchB.
  - Select the **Set Object Color** trigger option and change the properties as follows:
    - Change the **Object** property value from the default **item** to **current** by using the drop-down menu.
    - **Change the Color** property value from **Color**.random**()** to **Color**.green by using the drop-down menu.

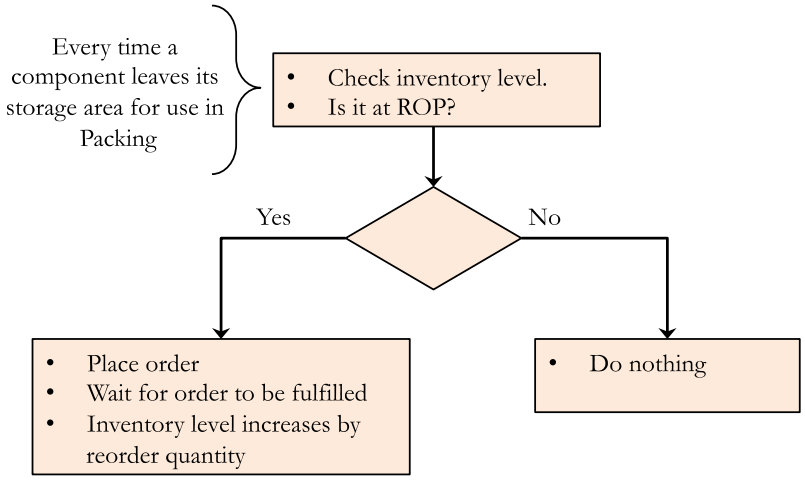


- Disconnect both component Sources. Since the Sources are being replaced by Process Flow logic, they could be deleted, but for now just disconnect them. As indicated earlier, in future models some of the components may be supplied based on a schedule; if so, the sources would be used. For this primer, all components are supplied via a reorder-point inventory system.

If you haven't already done so, save the model. Recall, it is good practice to save often.

3.2 Implementing reorder-point inventory logic using Process Flow

The basic logic for a reorder-point inventory system is shown in the figure below. The logic is contained in three sections, as denoted by the shaded boxes. Basically, every time a component is used in Packing the inventory level is checked. If the inventory level is not at the reorder point, no action is taken. However, if the inventory level equals the reorder point, the reorder quantity is ordered. There is a delay until the order is fulfilled; then, the level of inventory is increased by the number of items in the reorder quantity.



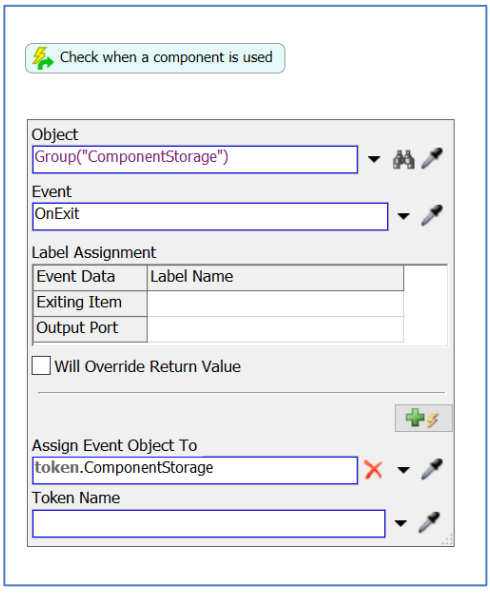
This logic is now implemented in *FlexSim* via Process Flow in three basic sections: Check for Reorder, Reorder, and Do Not Reorder.

3.3.1 "Check for Reorder" logic

The first set of logic involves three activities: Event-Triggered Source in the Token Creation section of the Library and Assign Labels and Decide in the Basic section of the library.

- Therefore, drag out these three activities onto the Process Flow workspace named Reordering

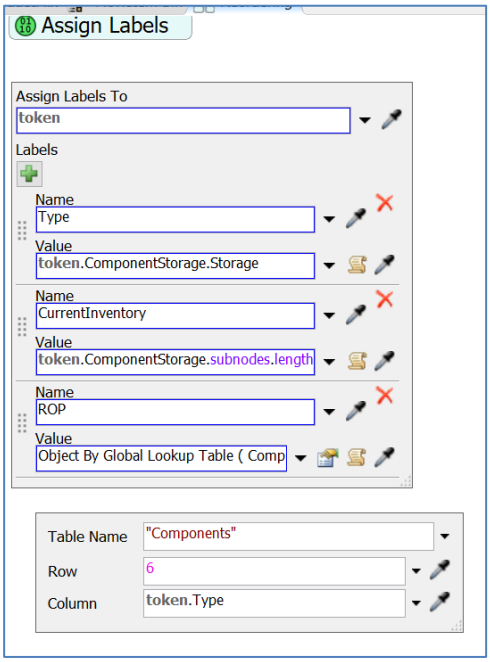
The first activity, Event-Triggered Source, is used to "listen" for any On Exit events that occur in any of the Queues that are included in the Group named ComponentStorage. This is implemented as described below and as shown in the figure to the right.



- Select the activity, and in the Quick Properties window, rename the activity **Check when a component is used**.
- The activity box can be resized as needed to make the text more readable by dragging one of the handles (small black squares) on the perimeter of the shape.
- The **Object** property is set by selecting **Group** from the drop-down menu, then selecting **ComponentStorage**. *FlexSim* writes the command **Group("ComponentStorage")** that obtains the property's value.
- The **Event** property is **OnExit**, selected from its drop-down menu.

- The value of the property **Assign Event Object To** is set to **token.ComponentStorage.** This action creates a label on the token that points to the Queue object from where the flowitem exited.

The next activity, Assign Labels, is used to obtain some information about the container that just exited the Queue and store it on the token. Each label is added by pressing the +. All of the labels are shown in the figure to the right.



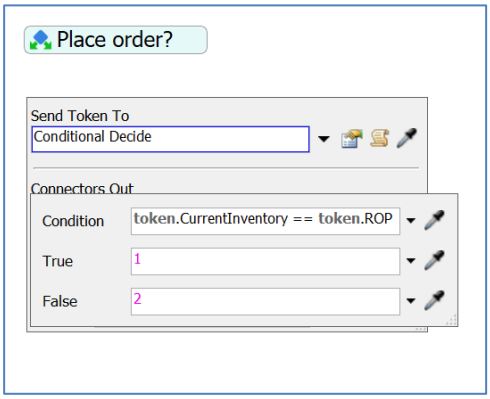
- The first label's **Name** is **Type**, and its **Value** is **token.ComponentStorage.Storage**. This expression can be typed in or the token.ComponentStorage portion of the expression can be obtained from the drop-down list of **Token Labels** and the ".Storage" portion can then be added at the end.

  This label contains the value of the label named Storage that is on the Queue object from where the flowitem just exited. Recall that ComponentStorage points to the Queue object where the item/container was stored.
- The next label is given the **Name** of **CurrentInventory** and its **Value** is set to **token.ComponentStorage.subnodes.length**. This label contains the value that is the number of items in the object from where the flowitem just exited (current contents of the object). subnodes.length is a *FlexSim* method that provides this information - how many items are currently in an object.

- The final label's **Name** is **ROP** and its **Value** is set by using the drop-down menu options **Table,** then **By Global Table Lookup.** The property values are set as follows and are shown at the bottom of the figure:
  - **Table** is set to **Components** using the drop-down menu.
  - **Row** is changed from the default value **token.labelName** to **6.**
  - **Column** is changed from the default value **1** to **token.Type** using the Labels drop-down option.

  This label stores the reorder-point value that would trigger an order; its value is obtained from the Components table and is based on the type of component.

The final activity in the Check for Reorder section is Decide, which is located in the Basic section of the Process Flow Library. Its properties are set as shown in the figure to the right.



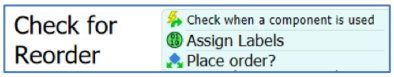
- Name the activity **Place order?**
- Set the property **Send Token To** to **Conditional Decide**.
- Set the **Condition** to:
  token.CurrentInventory == token.ROP.
  This can be typed in or the token.CurrentInventory == 1 option can be selected from the **Labels** drop-down menu and then edited; i.e., change **1** to **token.ROP**.
  This statement compares the number of items in the Queue where the flowitem/component just exited to the reorder point value for that type of component. If the two values are equal, the condition is True and the token will take branch 1 from the Decide activity and

an order will be placed. If the two values are not equal, the condition is False and the token will take branch 2 from the Decide activity and no further action is taken.

Note the use of == in the Condition expression. Since this is a comparison, the == operator must be used. A single = is an assignment operator that assigns the value of the expression to the right of the = to the variable named on the left side of the =.

Also, the connections from the decide activity to the two alternative blocks will be made after the blocks are created.

As shown in the figure to the right:



- Move the three activities so that they snap together in a sequence, or block.
- Using the Text activity (in the **Display** section of the Process Flow Library), annotate this section of logic to indicate what it does, e.g. **Check for Reorder**.

If you haven't already done so, save the model. Recall, it is good practice to save often.

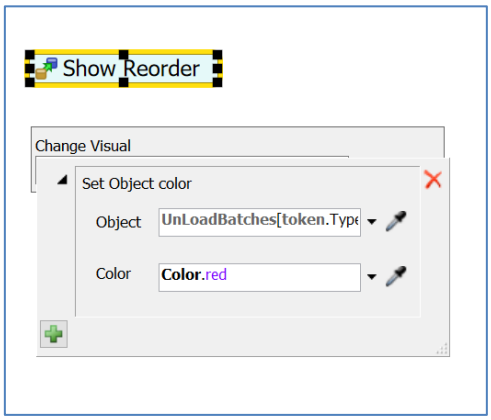
3.3.2 "Reorder" logic

If a component's inventory level is at its reorder point, then:

1. a delay is incurred that simulates the order replacement time, (
2. an order is created, and
3. the resulting order is unloaded by the finish operator.

The logic below implements this and changes the color of the batch area to red to indicate that an order has been placed. The color is reset to green once the order is fulfilled.

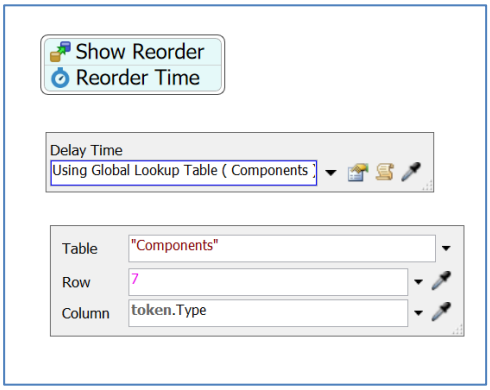
The Change Visual activity sets the appropriate Separator's (BatchA or BatchB) color to red to indicate that its component has been ordered. This is shown in the figure to the right and described below.



- Select the Change Visual activity from the Visual section of the Process Flow Library.
- Name the activity **Show Reorder**.
- For the **Change Visual** property, select using the + button the **Set Object Color** option, then
  - Set the **Object** property to **UnLoadBatches[token.Type]**. As you type this expression, *FlexSim* provides a list of possible values, one of which is the Global Variable named UnLoadBatches, which can then be selected. Then after the [token. is entered, then token.Type can be selected from the list of suggested items on the drop-down menu.

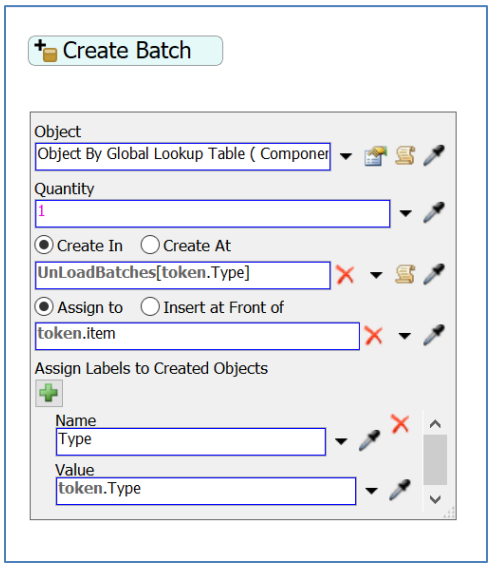
    This value is an Object property that points to to a Separator objects that is stored in the arrayed Global Variable named UnLoadBatches. The value is based on the component type, which is stored in the token's label named Type.
  - Set the **Color** property to **Color.**red which results from choosing Color.red on the drop-down menu.

The **Delay** activity (from the Basic section of the library) is used to set the reorder time. The token is delayed by the value stored in the Global Table Components. As shown in the figure to the right:



- Name the activity **Reorder Time**.
- Set the **Delay Time** using the drop-down menu option **By Global Table Lookup** with the **Table** parameter set by selecting "Components" from the drop-down list of model tables, the **Row** parameter value set to **7**, and the **Column** parameter set to the value stored in the token's label that is named Type, i.e. **token.Type**. Thus, the reorder time is the value stored in row 7 of the Global Table Components and in the column that corresponds to the value of the token's Type label.

The **Create Object** activity (in the Objects section of the library) creates a flowitem in the appropriate Separator that represents a component order. That flowitem will later be split into the number that is specified as the reorder quantity. This is very similar to the Create Object activity in the Initial Inventory section of the logic. As shown in the figure to the right:



- Name the activity **Create Batch**.
- The **Object** property is set using the **Flowitem by Global Table Lookup** drop-down menu option. The token creates a flowitem of the class that corresponds to its location (its number) in the list of the flowitems in the FlowItem Bin. In this case CompA and CompB are the 10th and 11th items in the list, respectively. These reference values are stored in the Components table. Therefore:
  - **Table** is set to **Components** from the list of Global Tables.
  - **Row** is set to **8**.
  - **Column** is set to **token.Type.** (This valued can be selected from list of tokens in the drop-down list **Labels**.) The column in the table depends on the value of the token's Type label (type of component).
- The **Quantity** value is **1** (the default value). This activity only creates an order. The order size is handled by the Separator.
- The value for **Create In** is **UnLoadBatches[token.Type]**. The flowitem is created in the object referred to in the arrayed Global Variable UnLoadBatches and depends on the value of the token's Type label (type of component).
- One label on the flowitem is created using the + button in the **Assign Labels to Created Objects** section of the interface.
  - Change the label's **Name** from labelName to **Type.**
  - Change the **Value** property from 0.0 to **token.Type** by selecting **Token Label** from the drop-down menu and then selecting **Type**. In this case, the value is either 1 or 2.

Another Change Visual activity is used to reset the appropriate Separator's (BatchA or BatchB) color, i.e., from red to green in order to indicate that its component's order has been delivered. The activity is the same as the Show Reorder activity.

- Select the activity, Change Visual from the Visual section of the Process Flow library.
- The activity name is **Show Delivery**.
- Select Set Object Color and the properties to:
  - **Object** to **UnLoadBatches[token.**Type**]**.
  - **Color** to **Color.**green from the drop-down list of colors.

- Drag out a Sink activity (from the Basic section of the library).
- Ensure all five of the activities created above are snapped together to form a "block" of activities.

3.3.3 "No Reorder" logic

This branch includes the activities that are executed by a token when the use of a component in the packing area does not trigger a reorder event. The branch could be composed of just a Sink, but an arbitrary 10-minute delay is added so the user can see the token route to this branch. There is no effect on the operation or performance of the model.

- Drag out a Delay activity and set the **Delay Time** property to **10**.
- Drag out a Sink activity and connect it to the Delay activity created above.

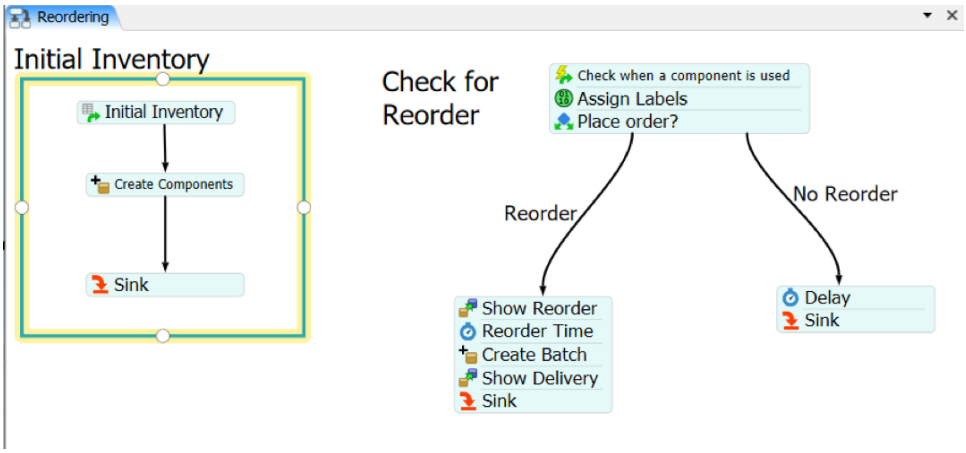
The "Reorder" and "No Reorder " blocks of activities are alternative responses to the "Place order?" Decide activity. Therefore, they must be connected to the activity, but the "Check to Reorder" block must be connected first.

- Click the bottom of the "Check to Reorder" block (the last activity in the block is "Place order?"). The cursor changes to a chain and as the mouse is moved an arrowed line connector appears. Complete the link by clicking on any side of the "Reorder block."
- Repeat the above step for the "No Reorder" block of activities.

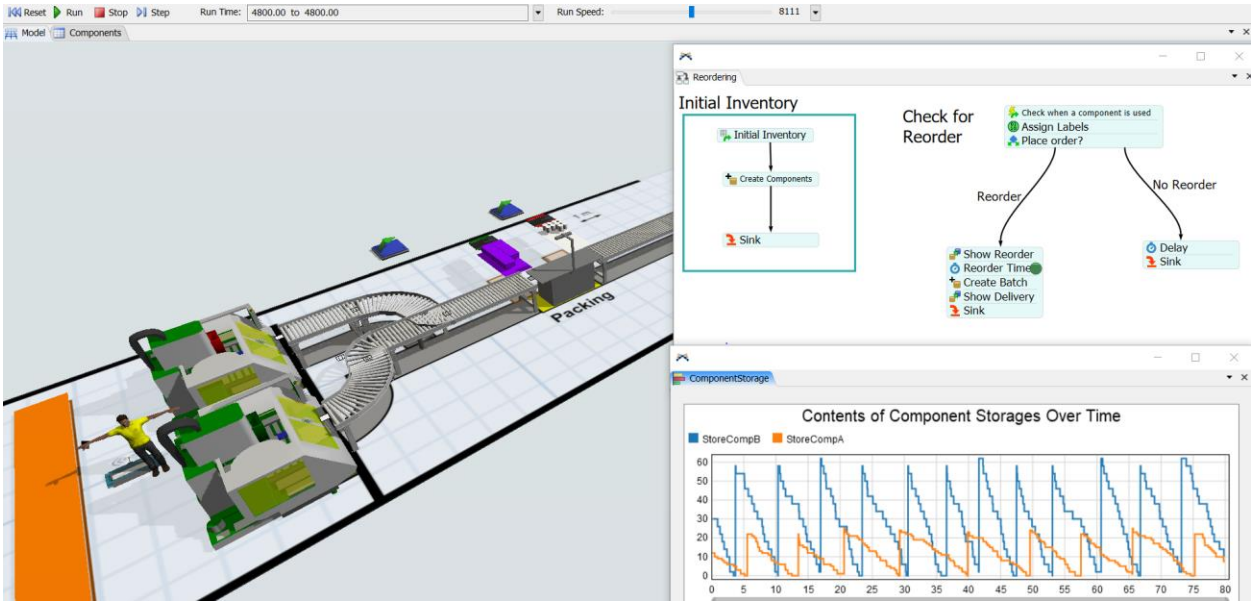
Notice the 1 and 2 labels at the bottom of "Check for Reorder" block. If the condition in the "Place Order?" activity is True (current inventory is equal to the reorder point for that component), its value is 1 and the "Reorder" block of activities are executed. Similarly, if the condition in the "Place Order?" activity is False, its value is 2 and the "No Reorder " block of activities are executed.

- Double-click the arrows and use the text box to label the connectors – **Reorder** and **No Reorder**, respectively. The size, color, and font of the text can be edited through the Quick Properties interface. Also, the text boxes can be moved for better placement.

The final Process Flow logic should look like the figure below.



The figure below is a screenshot of the current model running near the end of the prescribed Run Time. It includes the Process Flow logic and a Dashboard showing the contents of the two component Queues over time.



The Process Flow logic can easily be resized to fit the window size by right-clicking on the Process Flow workspace, selecting **View**, then selecting **Size to Fit**.

If you haven't already done so, save the model. Recall, it is good practice to save often.

4 CUSTOM TASK SEQUENCES IN PROCESS FLOW

This section provides a second extension of the primer model using Process Flow. In this case, a custom task sequence is developed using Process Flow to add a task to the finish operator's basic activity of moving containers from storage to finish machines for processing. The new task is that the operator must log information at a kiosk after a container is loaded onto a finish machine.

Prior to developing the logic in Process Flow, some modifications are made to the base model. Also, a simple study model is used to understand how to customize task sequences before implementing the logic in the main model.

4.1 Base model modifications

Some of the modifications in this section are to set up the change in tasks on the finish operator. Other modifications provide an opportunity to review and be introduced to new features in the 3D portion of *FlexSim*, such as:

- increase the production rate to the finish area,
- change from the A* algorithm back to using path networks to control the finish operator's travel paths,
- integrate the kiosk into the network, and
- hide the underlying layout that was used to build the model.

Begin with the basic model from the previous section, named Primer_4-2, and **Save As** Primer_4-3a. Basically make a copy of the model so that it can be customized and the original model is retained.

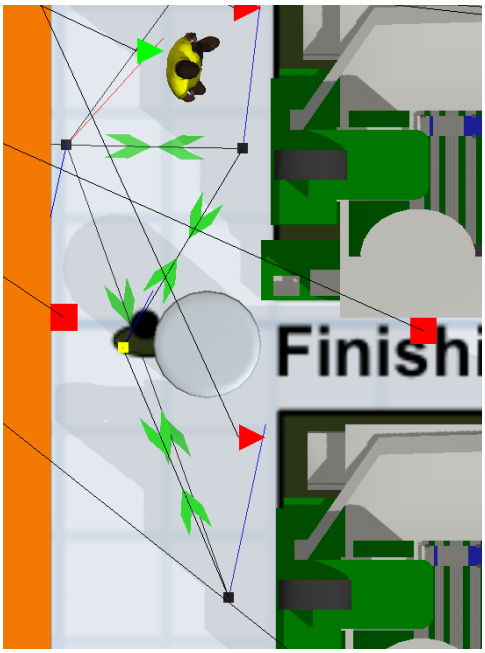
➢ Be sure the queue ContainersStorage's **Maximum Content** property is set to 5 (based on the experimentation in Section 4 of Part 3.

Based on running the simulation model, the utilization of the finish operator is only about 20% and the finish machines' utilizations are only about 60-65%. Therefore, it has been decided to increase the production rate by 25%.

For the production rate to increase by 25%, the average time between arrivals must decrease by 25%. The current inter-arrival time distribution is triangular(10, 35, 15), which has an average of 20 minutes. Therefore, the mean needs to decrease to 15 minutes. Assume the revised distribution is triangular(5, 30, 10), which has an average time between arrivals of 15 minutes.

➢ Update the triangular distribution's parameters in the Source, ContainersArrive.

➢ Confirm that the planned product mix is 20% for Type 1, 30% for Type 2, and 50% for Type 3 (recall, these are in the Global Table ProductMix).



Another task is going to be added to the operator – after the finish operator loads a container onto a finish machine, information needs to be logged at a kiosk that is located between the two machines. This task is added in the next section, but the kiosk object is added here, as described below and as shown in the figure to the right. It may be easier to add the object in planar view. (Recall, this is done by right-clicking on the modeling surface, selecting **View**, then **Reset View**.)

- To represent the kiosk, use the default Shape object, in the Visual section of the object Library. Retain its default size, a 1-meter high cylindrical object that is 1 meter in diameter.
- Name the object **Kiosk**.
- Position it between the two machines, say at **x = -8.5**, **y = -0.25**, and **z = 0**.
- Add the kiosk object as a barrier in the A* Navigator object in the **Members** portion of the Setup tab. Select FR Members, then use the + button to add the Kiosk, which is in the Visual Tools section of the object list.
- Hide the A* components, since a path network will be used. To do so, on the Visual tab of the A*Navigator, be sure all boxes are unchecked except for **Show Barriers** so that only the walls are displayed. If A* is used, the objects are still considered barriers even if hidden; not showing them just hides the blue shadowing under the object that indicates it is a barrier.

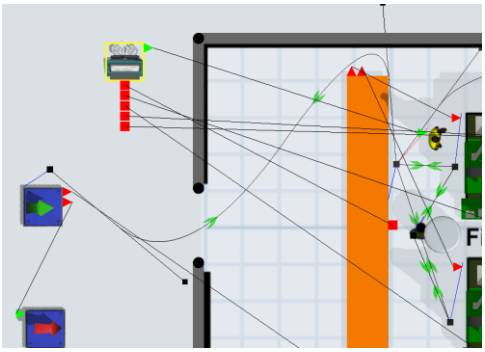
In this version of the model the path network will be used rather than the A* algorithm to control the finish operator's travel. Once the operator is connected to the network, it takes precedence over A*.

Make the following modifications to the path network.

- It may be easier if the Dispatcher object is moved out of the way; its placement does not affect the model.
- Also, it will be easier to do the remaining steps if the network is made visible again. Therefore, right click on the visible Network Node near the Queue ContainerStorage, then select **Network View Mode**, and then **Show All**.
- Add the operator to the network with an A-connection to the node near the Queue, ContainersStorage. (Recall, after the connection, a red line should show the connection between the two objects.)
- Change the Operator's property **Use navigator for offset travel** to **Travel offsets for load/unload tasks**.
- Integrate the kiosk into the path network.
  - Add a Network Node for the kiosk; name it **nn_Kiosk**.
  - Connect the Node to the kiosk object.
  - Disconnect the path between the Nodes at the finish machines, nn_FinMach_1 and nn_FinMach_2.
  - Connect nn_Kiosk to nn_FinMach_1 and connect nn_Kiosk to nn_FinMach_2, as shown in the figure above.

- Right-click on one of the green arrows on the network edge that leads to the Break Area. From the popup menu, change the edge property to **Curved**. Adjust the path around the Queue and through the door, as shown in the figure to the right, by dragging the handles. This is the same process that was used earlier to create the path to the packing area.

  Additional nodes could be used to form the path, but the edge between two nodes is a Bézier curve and is quite flexible.



- Hide all of the network nodes except the one by ContainersStorage. To do so, as before, right-click on that node, select **Network View Mode**, then select **None**. All nodes and edges will be hidden except for the one by the Queue. The network can be viewed again by following the same process except selecting either **Edges** (which only shows the edges; no nodes are shown) or **Show All** (all nodes and edges are displayed).

- Hide the layout on which the model was built. To do so, open the Tree and then double-click on the Layout object to open the user interface. In the Flags section of the General tab, uncheck the **Show 3D Shape** box. This way the layout can be displayed again by reversing this process.

If you haven't already done so, save the model. Recall, it is good practice to save often.

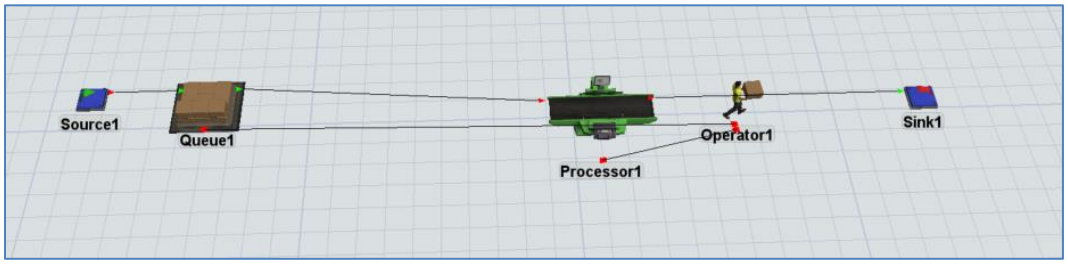
4.2 Simple study model of task sequences in Process Flow

Before adding the information-logging task to the finish operator in the main model, develop a simple study model in order to understand some basic concepts of working with Task Executers. As has been stressed in this primer, it is good modeling practice to test out new concepts and more advanced logic in a study model before implementing it in the primary model. This allows the modeler to focus on the salient aspects of the change.

Start with a new FlexSim model, accept the default model units, and name it Simple TE Process Flow.

Begin with the most basic model, as shown below, with all default values. The operator is used to transport items from the Queue to the Processor and from the Processor to the Sink.

- Drag out all of the required objects and connect them as shown in the figure below.
  - In addition to A-connecting the Fixed Resource object, be sure to S-connect (Center Connect) the Queue and Operator and the Processor and Operator.
  - Also, be sure to check the **Use Transport** box on the Flow tab on both the Queue and Processor.



For comparison, two similar models are considered. To facilitate this, we will make a copy of the above model. This is a convenient way to duplicate sets of objects.

- Select all of the objects – hold down the Shift key and use the left mouse button to encompass all of the objects; this results in a gray box that contains the objects. When the mouse button is released, all of the objects should be selected; i.e., the outline of all objects should be red.
- Copy, using Ctrl-C, the selected objects, then click on the modeling surface somewhere below the current model, and using Ctrl V, paste the set of objects. The selected set can be moved as a group to position them on the modeling surface. Once set, unselect all of the objects by holding down the Shift key and clicking with the mouse anywhere on the modeling surface (not on any objects).

The two models in the file are completely independent since there are no flows, resources, events, or activities that are shared. In this exercise the upper model will remain as is; it is there just for comparison. The lower model will be modified to use Process Flow to control some of the operator's tasks. The models can be used to compare task sequences using the default 3D approach and customizing them through Process Flow.

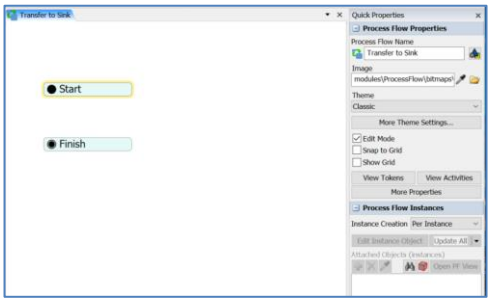
A Shape object, like the kiosk in the main model, is used in this example. The operator will carry each flowitem to it once processing is finished at the Processor. At the Shape object, the operator and flowitem are delayed for additional processing. After the delay, the operator carries the flowitem to the sink and unloads it.

Note that the transport from the Source to the Processor remains unchanged. It is managed by the 3D model (in the Flow tab of the Source) and not by Process Flow logic. There is no need to modify it since the default logic is fine.

If you haven't already done so, save the model. Recall, it is good practice to save often.

- Add a Shape object, from the Visual section of the Library, anywhere in the second model, somewhere between the Processor and Sink. Again, this is like the kiosk in the main model.

In order to have the operator travel to and be delayed at the Shape object before being unloaded at the Sink, Process Flow is used to replace the basic task sequence for transport with a custom task sequence.

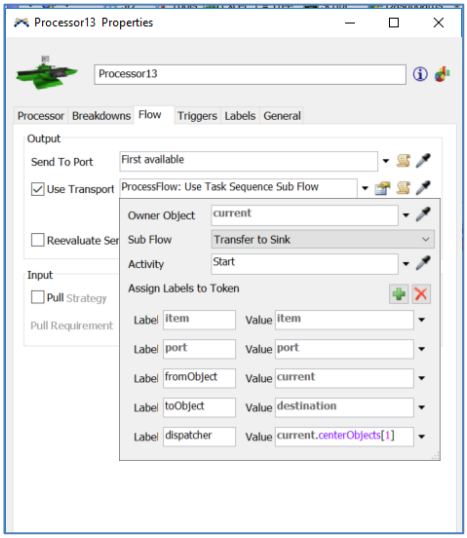


Start the Process Flow logic as shown in the figure to the right and as described below.

- From the Process Flow icon on the Main Menu, or via the Toolbox, add a **Sub Flow** to the model.
- In the **Process Flow Properties** section of the Quick Properties window, change the **Process Flow Name** from SubFlow to **Transport to Sink**.
- In the **Process Flow Instances** section of the Quick Properties window, change the **Instance Creation** property value from **Global** to **Per Instance**.
- From the Sub Flow section of the Process Flow Library, drag out Start and Finish activities onto the Process Flow workspace.

FlexSim's default transport logic using a Task Executer, as was described in Section 3 of Part 2, has the Operator perform the following uninterrupted sequence of tasks. In this primer, this set of four tasks is referred to as the Transport Task Sequence.

1. Travel to the object where the flowitem is located; the object is referred to as fromObject.
2. Load the flowitem from the fromObject.
3. Travel with the flowitem to its destination object that is referred to as toObject.
4. Unload the flowitem to the toObject.

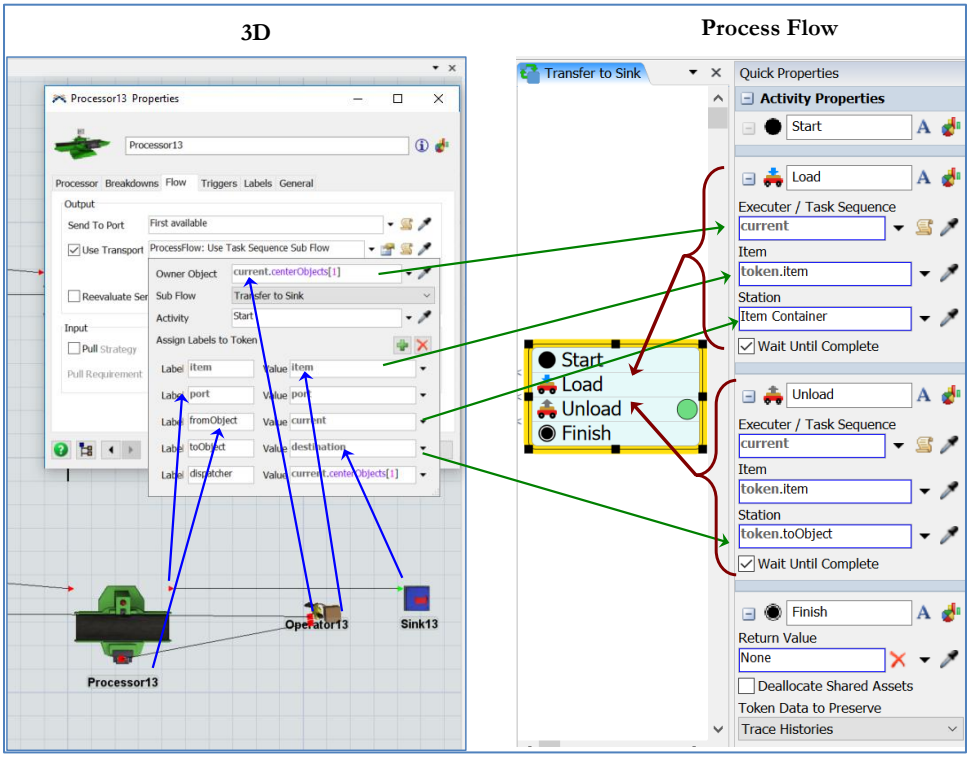


These four tasks will now be handled by Process Flow so that they can be easily modified and customized. To pass the responsibility for transporting a flowitem from the Processor, the **Use Transport** property on the Flow tab of the Processor is changed, as described below and shown in the figure to the right.

- On the Processor's Flow tab, the Use Transport box should already be checked. Change the default **current.**centerObjects[1] to **ProcessFlow: Use Task Sequence Sub Flow** via its drop-down menu.
- Change the resulting interface as follows:
  - Change the **Owner Object** property from **current** to **current.**centerObjects[1] by selecting it from the **Connected Objects** option on the property's drop-down menu.
  - All of the remaining properties are the default values. However, be sure the **SubFlow** property is **Transfer to Sink** (the name of the newly created Process Flow Sub Flow) and the **Activity** property is **Start**. The remaining items in the interface is information that is sent to Process Flow about the Processor object and the flowitem that needs to be transported.

As shown in the following figure, information needed to perform the transport in the 3D model is captured in the Processor's Flow tab and sent to Process Flow. The sending of this information creates a token in Process Flow and that token contains all of this information in its labels. The token flows through a set of activities that models the transport operation. The token's labels are defined as:

token.item	flowitem that needs to be transported.
token.fromObject	object that the flowitem is transported from; i.e., Processor 13 for the case shown in the figure.
token.toObject	object that the flowitem is transported to; i.e., the Sink for the case shown in the figure.
current	the task executer that will perform the transport. It is the object that is connected to the center port of Processor13, current.centerobject[1]; i.e., Operator13 for the case shown in the figure.



As shown in the figure above, the basic transport is accomplished in Process Flow by using two activities Load and Unload, both found in the Task Sequences section of the Activity Library. In the 3D portion of the figure, the Operator is carrying the item to the Sink and correspondingly the token in Process Flow is in the Unload activity.

The Load activity is actually a small task sequence, carried out by, in this case, Operator13. The task sequence contains the first two tasks in the Transport Task Sequence thast was defined above – travel to the fromObject and load the item. Similarly, the Unload activity is also a task sequence, again carried out by Operator13 in this case, that contains the last two tasks in the Transport Task Sequence defined above – travel to the toObject and unload the item. In both models, the task sequence is considered complete with

the unloading of the flowitem at the Sink; therefore, the Operator waits at the Sink for the next task sequence to be assigned.

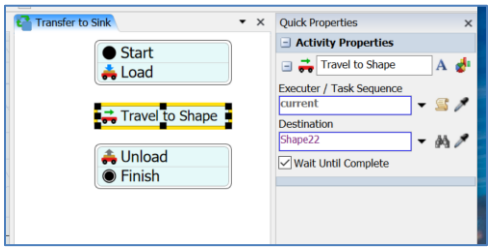
Add the Load and Unload activities in the Sub Flow and modify their properties as shown in the figure above.

- Drag a Load activity from the Task Sequences section of the Process Flow Library.
  - Change the **Executer / Task Sequence** property to **current** by selecting **current (Instance Object)** from the drop-down menu.
- Similar to the previous step, drag an Unload activity from the Task Sequences section of the Process Flow Library.
  - Change the **Executer / Task Sequence** property to **current** by selecting **current (Instance Object)** from the drop-down menu.
  - Change the **Station** value from **token.**destination to **token.**toObject**.**
- Join the four activities to form a logic block that is the sequence: Start, Load, Unload, Finish.

- **Reset** and **Run** the model. Confirm that both model segments operate similarly. One model uses *FlexSim*'s default task sequence for transporting items by a task executer; the other model uses a custom task sequence created in Process Flow.

If you haven't already done so, save the model. Recall, it is good practice to save often.

The new task, to travel to the Shape object, is added as shown in the figure to the right and described below.



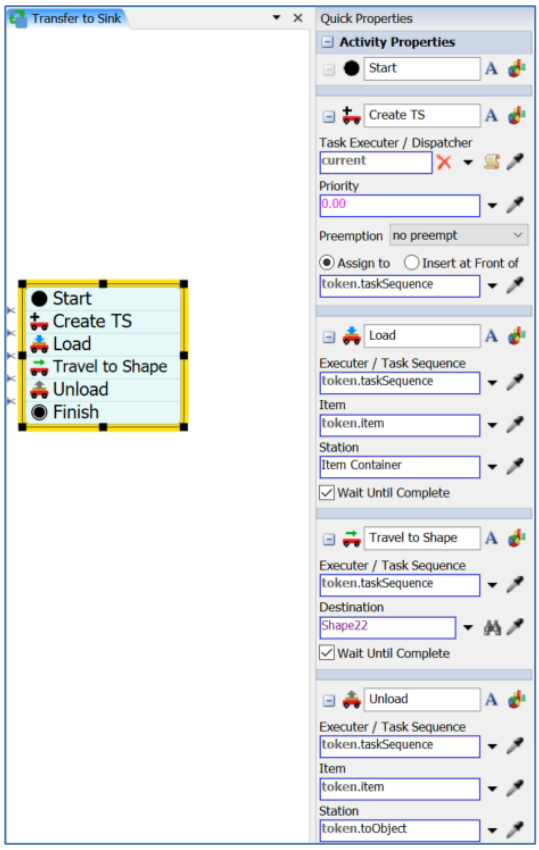
- Select the block of activities and click on the "scissors" symbol between the Load and Unload activities in order to break the block so a Travel activity can be inserted.
- Drag a Travel activity to Process Flow from the Task Sequences part of the Library, and:
  - Change the **Executer / Task Sequence** property to **current**. As before, select **current (Instance Object)** from the drop-down menu.
  - Change Destination property to the name of the Shape object, Shape22 in this case. (Note that the number in the object names may be different since in this study model, the default *FlexSim*-generated names are used.) The Shape object can be selected either by:
    1. the property's drop-down menu **FlexSimEventHandler** and selecting the desired object or
    2. clicking on the object in 3D using the property's Sampler tool.
- Change the name of the task from the default Travel to **Travel to Shape**.
- Insert the Travel activity into the logic block between Load and Unload. Snap all activities together to form one block.

- **Save**, **Reset**, and **Run** the model. Check to be sure to model works as planned. Note that it does **not** work as planned! Here's what happens after an item is finished at the Processor:
  1. Operator travels to the Processor and loads an item →Correct.
  2. Operator travels to the Shape object →Correct.
  3. Operator travels to the Source and loads another object; the operator now has two objects → Incorrect.
  4. Operator travels to the Processor and unloads the item from the Source → Correct.
  5. Operator travels to the Sink and unloads the object → Correct.

The problem is there are now four task sequences: the default Transport Task Sequence (from the Source to the Processor) plus the three Process Flow sequences: , Load (after Processing), Travel (to Shape), and Unload (at Sink). As flowitems enter the system and finish processing their resulting tasks are sent to the Task Executer and are queued in a First-In, First-Out manner. The Operator is processing tasks in the order in which they are received; however, this results in an operation that is not what is desired.

To remedy the problem, all three Process Flow task sequences need to be processed as one and not as separate task sequences. The Process Flow logic is modified as shown in the figure to the right and described below.

- "Cut" the logic block and add the Create Task Sequence activity (name is Create TS) between the Start and Load activities. Rejoin the activities, again into one block.
- Change the Create TS activity's **Task Executer / Dispatcher** property to **current**. Reference to the task sequence is stored in the token's label named **taskSequence** (token.taskSequence).
- Change the **Task Executer / Dispatcher** property in all of the remaining activities (Load, Travel, and Unload) from **current** to **token.taskSequence**. Each of these can be selected from the drop-down menu **Token Label**.



- **Save**, **Reset**, and **Run** the model. Verify that the model works as planned. It should!
  The model is correct, if the Operator transports only one flowitem at a time; and, after the operation at the Processor is complete, the Operator should take the item to the Shape object and then to the Sink, where it is unloaded.

If you haven't already done so, save the model. Recall, it is good practice to save often.

4.3 Custom task sequences in the primer model

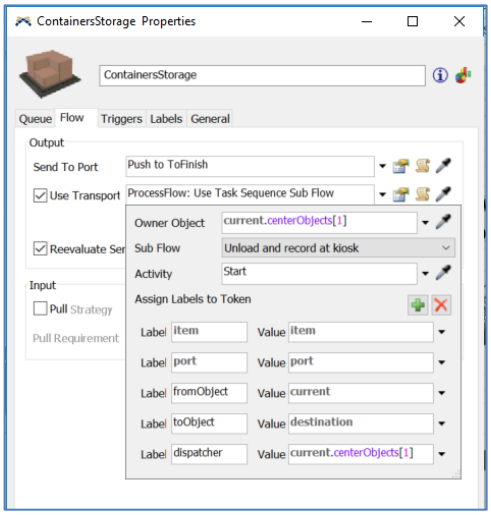
The concepts developed in the previous section are now applied to the main primer model. The Process Flow logic is very similar to that described in the simple study model.

Begin with the basic model named Primer_4-3a, and **Save As** Primer_4-3b. Basically make a copy of the model so that it can be customized and the original model is retained.

Start the Process Flow logic as described in the previous section.

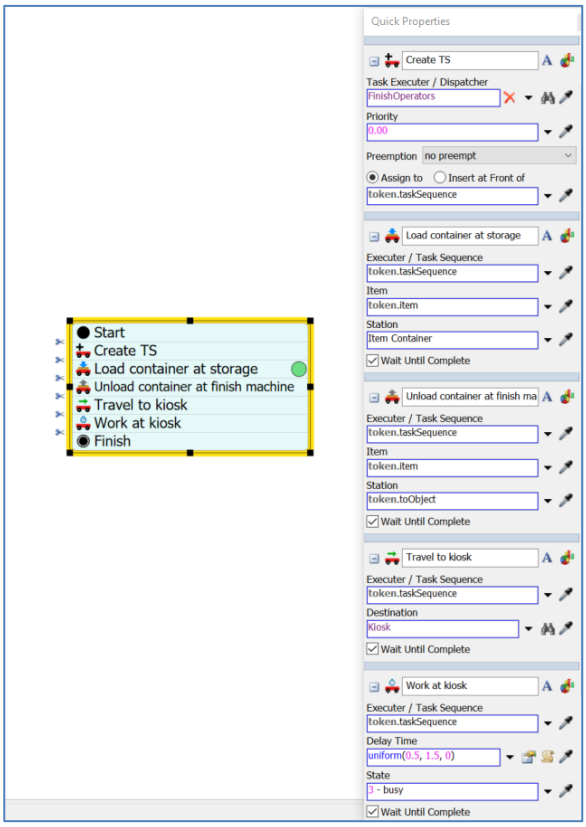
- From the Process Flow icon on the Main Menu, or via the Toolbox, add a **Sub Flow** Process Flow logic to the model.
- In the Quick Properties interface, change the **Process Flow Name** from SubFlow to **Unload and record at kiosk**.
- In the Quick Properties interface, change the **Instance Creation** property value from **Global** to **Per Instance**.
- From the Sub Flow section of the Process Flow Activity Library, drag out Start and Finish activities.

The responsibility for defining the tasks to transport containers from the Queue ContainersStorage to a finish machine is changed from the 3D object to Process Flow logic so that the finish operator can travel to the kiosk after unloading a container at the finish machine.



- On the Queue's Flow tab, the Use Transport box should already be checked. Change the default **current.**centerObjects[1] to **ProcessFlow: Use Task Sequence Sub Flow** via its drop-down menu.
- Change the properties on the resulting interface as follows:
  - Change the **Owner Object** property from **current** to **current.**centerObjects[1] by selecting it from the **Connected Objects** option on the property's drop-down menu.
  - All of the remaining properties are the default values. However, be sure the **SubFlow** property is **Unload and record at kiosk** (the name of the newly created Process Flow Sub Flow) and the **Activity** property is **Start**. The remaining items in the interface are information that is sent to Process Flow about the Queue object and the flowitem that needs to be transported.

Define the activities for the Process Flow and as each activity is defined add it to the logic block, as shown in the figure to the right.



- So that all of the tasks are performed as a group, start the logic flow with the Create Task Sequence activity.
  - Its default name, **Create TS**, is okay.
  - Change the **Task Executer / Dispatcher** property to the operator's Dispatcher object named **FinishOperators**. This can be selected from the Dispatcher section of the drop-down menu.
  - The default **Assign To** label name – **token.taskSequence** – is okay.
- Use the Load activity for the Operator to travel to the Queue and pick up a container.
  - Change its name to **Load container at storage**.
  - Change the **Executer / Task Sequence** property to **token.taskSequence**. from the Token Label section of the drop-down menu.
  - Change the **Station** property to **token.fromObject**. Recall this is passed from the Queue when requesting a travel resource.

1. Use the Unload activity for the Operator to travel, with the flowitem, to a finish machine and unload the container.
   - Change its name to **Unload container at finish machine**.
   - As in the previous activity, change the **Executer / Task Sequence** property to **token.taskSequence**.
   - Change the **Station** property to **token.toObject**
- Since longer, more descriptive activity names are used, it is advised that the size of the activities be changed for readability. To do so, select a side handle on the logic block and drag it horizontally to the desired width. All subsequent activities that are added to the block will be resized to this larger width.
- Use the Travel activity for the finish operator to travel to the kiosk. Change its properties as follows:
  - Change its name to **Travel to kiosk**.
  - As in the previous activity, change the **Executer / Task Sequence** property to **token.taskSequence**
  - Change the **Destination** property to **Kiosk** either by the drop-down menu option **FlexSimEventHandler** or by using the Sampler tool.
- Use the Delay activity for the finish operator to spend time at the kiosk. Be sure to use the Delay activity in the Task Sequences section of the Library and not the one in the Basic section. Change the activity's properties as follows.
  - Change the Delay activity's name to **Work at kiosk**.
  - As in the previous activity, change the **Executer / Task Sequence** property to the token label **token.taskSequence**

- Change the **Delay Time** property from the default value of 10.0 to **Statistical Distribution,** then **Uniform** by the drop-down menu. In the resulting interface, set the distribution's parameters: **Minimum** to **0.5** and **Maximum** to **1.5**. Therefore, the operator is delayed at the kiosk for a randomly-generated amount of time that is between 0.5 and 1.5 minutes. Note, the average delay time is one minute.

➢ **Reset** and **Run** the model. Verify that the finish operator travels to and is delayed at the kiosk each time after a finish machine is loaded, as shown in the figure to the right.

If you haven't already done so, save the model. Recall, it is good practice to save often.

A close examination of what happens during the simulation, which can be noted early on in the simulation via the **Step** feature on the Model Execution Toolbar, is that the Operator unloads a container at a finish machine, goes to the kiosk, then returns to the machine if a setup is required. This is because there are two task sequences at play here:

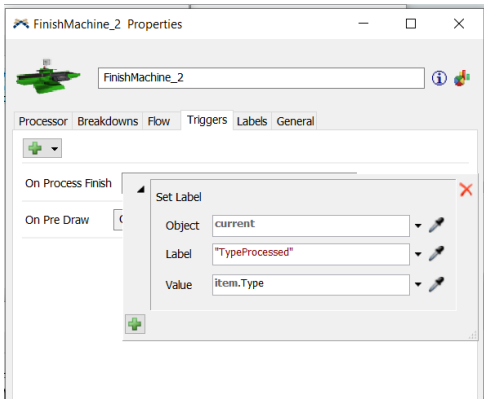
1. the load, unload, and kiosk work, as defined in the section above in Process Flow and
2. the setup operation using the Operator, if needed. This may be the way the real system works, or it may not.

In the case of this example, this is not the way the system works. The Operator should load a flowitem at the Queue, travel to the desired finish machine, unload the item, perform the setup operation, if needed, and then travel to, and work at, the kiosk. Therefore, the Process Flow logic needs to be modified to include the setup operation; i.e., the setup logic on the two Processors (finish machines) that was included in the model earlier in the primer via drop-down menu options needs to be replaced by Process Flow logic.

Begin with the basic model named Primer_4-3b, and **Save As** Primer_4-3c. Basically make a copy of the model so that it can be customized and the original model is retained.

Prior to modifying the Process Flow logic, a new object label needs to be added to both Processors that tracks the type of the flowitem/container that was most recently processed. This is used to determine if a setup is required or not - no setup is required if an arriving item is of the same type as the one that was just processed.

- On the Labels tab of each Processor (FinishMach_1 and FinishMach_2), use the + button to **Add Number Label**.
  - Change the name from labelName to **TypeProcessed**.
  - Check the box at the bottom of the interface **Automatically Reset Labels**. Whenever, the model is Reset, the label value will be set to 0. This will ensure that a setup always occurs when a simulation starts because the arriving item Type value will never be 0. (In this example, the value will be 1, 2, or 3.)
- On the Triggers tab of each Processor, use the + button to add an **On Process Finish** trigger.
  - Using the + button on the trigger, select **Data** and then **Set Label** on the drop-down menus.
  - Set the property values on the **Set Label** interface as follows and as shown in the figure to the right.
    - Change the **Object** property from the default item value to **current** using the drop-down menu. That is, set the label value on the machine, not on the flowitem.
    - Change the **Label** property from the default **"Type"** value to **"TypeProcessed"** using the drop-down menu.
    - Change the **Value** property from the default value to **item.Type** by selecting the value from the **Labels** section of the drop-down menu. This sets the value of the Processor's label named TypeProcessed to the value of the item's label named Type that just finished processing.



As was defined earlier in the primer, each finish machine (Processor) handles the logic for:

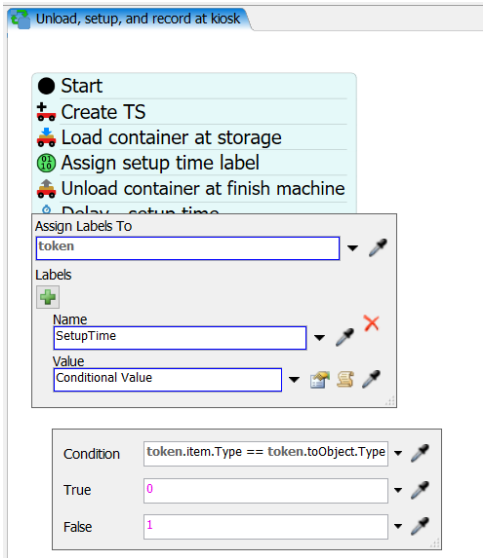
1. determining if a container requires a setup or not,
2. if a setup is required, determine the setup time, and
3. also if a setup is required, obtain an Operator to perform the setup.

All of this logic now needs to be removed from the 3D objects because it will now be handled through Process Flow logic.

- On the Processor tab of each Processor:
  - Change the **Setup Time** property's value that had been added earlier in the primer to **0** by selecting **No Setup Time (0)** from the drop-down menu.
  - Uncheck the **Use Operator(s) for Setup** box.

Define and incorporate the setup logic from the Processors into the recently created Sub Flow named **Unload and record at kiosk**.

- Modify the name of the Processor Flow to reflect the new logic; i.e., **Unload, setup, and record at kiosk**.
- Split the logic block after the activity **Load container at storage**.
- Add an Assign activity from the Basic section of the Process Flow Library. Its properties are set as described below and as shown in the figure to the right.



  - Name the activity **Assign setup time label.**
  - Change the token label **Name** to **SetupTime**.
  - Set the **Value** property to **Conditional Value** and change the property values on the resulting interface.
  - Set the **Condition** property value to:

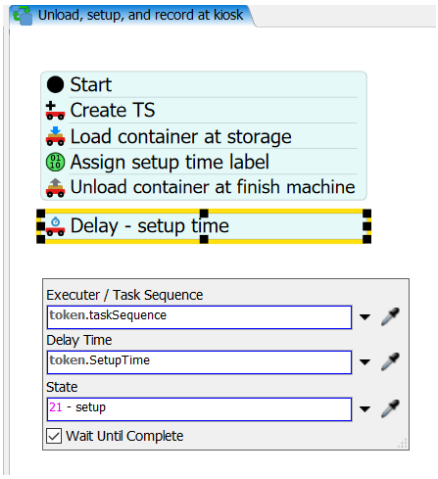
token.item.Type == token.toObject.TypeProcessed.

This statement compares the type of container/item that needs to be moved from the Queue to the type of container that was last processed at the finish machine where the container/item is being moved to.

If the two values are equal, then the value specified in the value of **True** property is the setup time. Conversely, if the values are not equal, the setup time is the value specified in the **False** property.

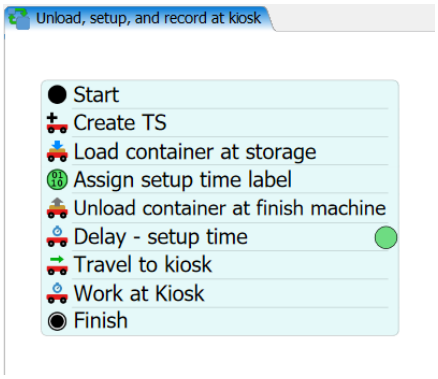
  - Set the value of the **True** property to **0** (no setup time is needed since the items are of the same type) and set the value of the **False** property to **2**. (setup time is 2 minutes).

- Add a Delay activity after the Unload activity, as shown in the figure to the right and as described below.



  - The value of the **Executer / Task Sequence** property is the token label **token.taskSequence**.
  - The value of the **Delay Time** property is the value of the token label that was set earlier in the Assign activity, **token.SetupTime**.
  - The value of the **State** property is **21 – setup**, selected from the dropdown menu.

The final Subflow Process Flow logic is shown in the figure to the right. It controls the Operator for the set of tasks that include: transporting flowitems from the container Queue to the finish machine Processors, performing a setup operation, if needed, and performing work at the kiosk.

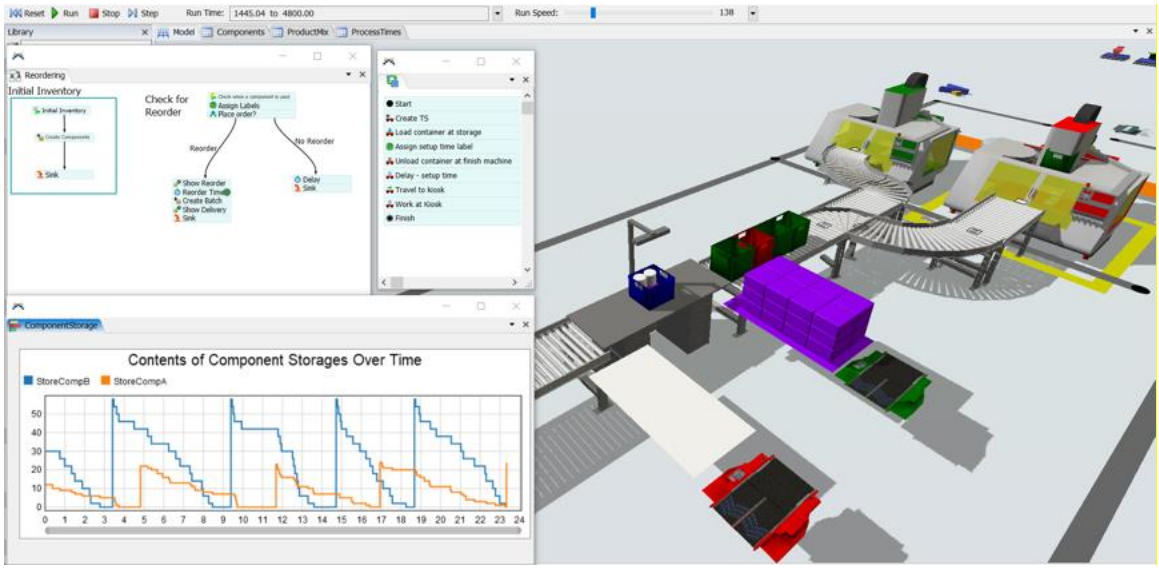


If you haven't already done so, save the model. Recall, it is good practice to save often.

- **Reset** and **Run** the model. Verify that the finish operator moves containers from the ContainerStorage object to the finish machines, performs a setup if needed, and then travels to and works at the kiosk before undertaking another task.

A screenshot of the final model as it is running is shown in the figure below; it includes:

- 3D view of the finishing and packing areas with a variety of fixed and mobile resources, downtimes, etc.
- Two applications of Process Flow - reorder-point inventory management and custom task sequence.
- Example Dashboard output, inventory levels of the two components; this clearly shows the dynamics of the system and the basic reorder-point means of managing inventory.



Note that this model can now be easily scaled up – add more finish machines, packing stations, operators, etc. These can be added by mostly copying and pasting the objects.

Also, if the modeling started with many finish machines or packing stations, then each change in the model would require the change be made in each of the objects. This is not only tedious, but subjects the model to potential errors. Therefore, *it is always good modeling practice to start small, add complexity as needed in small steps, test and verify, then scale* up.

This page is intentionally blank.